nmg.labs

NMG LABS · COMPUTER VISION HIRING

Image Matching Challenge

Feature Matching, RANSAC and Honest Results

A same-day, hands-on assignment. Build a Python tool that decides whether two photos show the same physical object, and prove it with numbers.

NMG Labs Team
Candidate brief · 5 to 6 hours, same day

ORIENTATION

Why This Matters

Show us how you solve a real image-matching problem in Python and OpenCV, from raw images to a decision you can defend with numbers.

Given two images, your tool must decide whether they show the **same physical object**, not merely the same kind of thing. Two marble tiles cut from the same block share colour and veining, but they are different pieces, and your tool must say so.

> TOTAL TIME
>
> About **5 to 6 hours, in one day**: from the morning briefing to the evening deadline. The exact deadline is announced at the briefing.

We use your submission to decide on the next interview round. We assess:

- Practical command of OpenCV: preprocessing, feature extraction, matching and RANSAC
- Whether your decisions hold up on lookalike images and on images you have never seen
- How you choose, explain and test your confidence score and thresholds
- Code quality, and how honestly you report what does not work

YOUR DAY · SAME-DAY ASSIGNMENT

Plan your 5 to 6 hours

YOUR DAY · PART A

How the day runs

WHEN	WHAT HAPPENS
Morning	Briefing call; this brief and the dataset are shared
During the day (5 to 6 hours)	You build, test and write up; ask questions in the shared channel
Evening deadline	Push your final code and report; commits after the deadline are ignored
After the deadline	We run your `evaluate.py` on our hidden pairs
Next round	Shortlisted candidates are invited to a technical interview

YOUR DAY · PART B

A suggested split

BLOCK	FOCUS
Setup and preprocessing	Load the data, resize, grayscale, test whether CLAHE helps
Features, matching, RANSAC	The core pipeline: requirements 2 to 4
Scoring and thresholds	Confidence score and the three bands
Evaluation and write-up	Run `evaluate.py`, write `REPORT.md`, push

Leave time for the write-up: an untested tool with no report scores poorly, however good the code is.

PART 1 · THE PROBLEM

Understand the task and the data

PART 1 · A

The big idea: same object, not same kind

This is the single most important idea in the assignment. Read it before you write any code.

> PLAIN-ENGLISH ANALOGY
>
> **Your tool must recognise a fingerprint, not a family resemblance.**

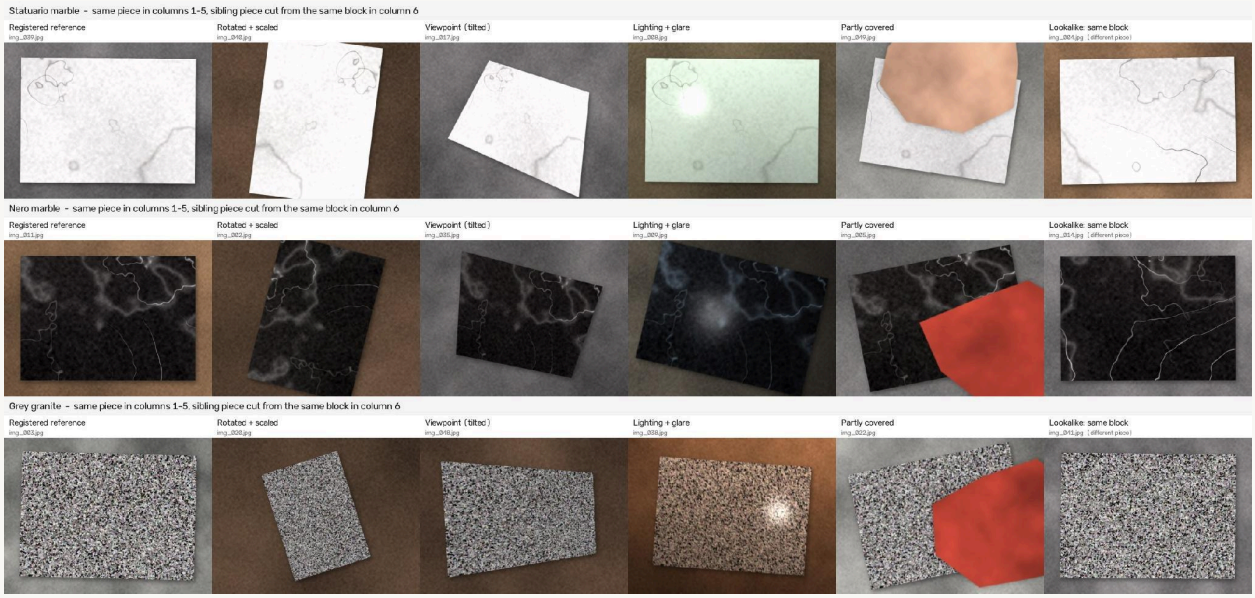
- **Same object:** the same physical piece, photographed again under different conditions.
- **Lookalike:** a different piece cut from the same block. Same colour, same vein style, different pattern.
- **Different:** a piece from another block.

Methods that compare overall colour or texture recognise the family, not the individual, and fail on lookalikes. Matching specific local points that must line up geometrically recognises the individual. That is what this assignment tests.

PART 1 · B

The dataset

You receive a labelled dataset in the morning. We keep a hidden set, built the same way with new pieces, to test every submission after the deadline.



Each row shows one piece under every condition. The last column is a different piece cut from the same block, which must not match.

PAIR TYPE	WHAT IT TESTS	LABEL	PAIRS
`rotation_scale`	The piece turned to any angle, photographed nearer or farther	same	10
`viewpoint`	The camera tilted, not square to the surface	same	10
`lighting`	Uneven light, colour cast, glare spots and some blur	same	10
`occlusion`	25 to 55% of the piece covered by another object	same	10
`lookalike`	A different piece cut from the same block	different	20
`different`	A piece from a different block	different	20

Download the dataset: NMG image matching dataset (Google Drive)

Folder layout

```
data/
  images/img_001.jpg ...    # 50 photos of stone pieces, 1280 x 960
  pairs.csv                 # pair_id, image_a, image_b, label, pair_type
  gallery/                  # bonus: 12 registered reference photos
  gallery_queries/          # bonus: 8 photos to identify
  gallery_queries.csv       # query_image, expected_match (or none), condition
```

`image_a` is always the registered reference photo and `image_b` the photo to check. File names are random and carry no meaning. Do not rename, crop or edit the provided images. You may add your own photos for extra testing, but report results on the provided pairs separately.

PART 2 · BUILD

Six requirements, one pipeline

PART 2 · A

The requirements

All six requirements are mandatory. Explain every choice you make in the brief document.

STEP 1	STEP 2	STEP 3	STEP 4	STEP 5	STEP 6
Preprocess	**Extract features**	**Match**	**Ratio test**	**RANSAC**	**Score**
Resize, grayscale, enhance	SIFT, ORB or AKAZE	BFMatcher or FLANN	Drop ambiguous matches	Inliers and inlier ratio	Same, Similar, Different

Only matches that survive the RANSAC check count towards the confidence score.

1. Image preprocessing

- Load images with OpenCV and handle a missing or unreadable file without crashing.
- Resize to a fixed long side, convert to grayscale, and apply basic enhancement (such as CLAHE) if it helps.
- Show whether your enhancement actually improves results; do not add it by default.

2. Feature extraction

- Implement at least one of SIFT (preferred), ORB or AKAZE. Other options are allowed if you justify them; see Approved alternatives below.
- Explain why you chose it, and report the number of keypoints found in each image.

3. Feature matching

- Match with BFMatcher or FLANN, using the right distance for the descriptor: L2 for SIFT, Hamming for ORB and AKAZE.
- Apply Lowe's ratio test to filter bad matches. State the ratio you used and why.

4. Geometric verification

- Validate matches with RANSAC (`cv2.findHomography`). A robust variant such as USAC_MAGSAC, RHO or LMEDS is also accepted; name the method you used.
- Count the inliers and calculate the inlier ratio: inliers ÷ good matches.
- Visualise the inlier matches only, not all matches.

- Handle pairs with too few matches for RANSAC (fewer than 4) without crashing.

5. Similarity scoring

- Generate a confidence score from 0 to 100% and explain the formula. Base it on RANSAC inliers (count and ratio), not raw match counts. It must be 0 when geometric verification fails.
- Define three bands and choose their thresholds from the labelled pairs:

BAND	MEANING
Same object	Strong geometric evidence that both images show the same physical object
Similar object	Some evidence, but not enough to prove it is the same object; treated as not proven
Different object	No consistent geometry between the images

A lookalike pair must never be classified as Same object. Explain how you chose each threshold.

6. Output

FOR EVERY PAIR, PRODUCE

- ☐ A matched-image visualisation showing the inliers
- ☐ Number of keypoints detected in each image
- ☐ Total matches
- ☐ Good matches after the ratio test
- ☐ RANSAC inliers and inlier ratio
- ☐ Final confidence score and band
- ☐ Processing time

Approved alternatives to SIFT and RANSAC

You may swap any stage of the pipeline. State what you used and why in REPORT.md, and compare it with SIFT and RANSAC on the provided pairs.

- **Feature detectors:** SIFT (preferred), RootSIFT (SIFT with Hellinger-normalised descriptors), AKAZE, KAZE, ORB or BRISK. ORB, AKAZE and BRISK are binary, so match them with Hamming distance. SURF is patented and missing from standard OpenCV builds, so do not use it.
- **Matchers:** BFMatcher, with or without cross-check (mutual nearest neighbour), or FLANN with a KD-tree index for SIFT and an LSH index for binary descriptors. Keep Lowe's ratio test with any of them.
- **Geometric verification:** cv2.findHomography with cv2.RANSAC, cv2.USAC_MAGSAC (MAGSAC++), cv2.USAC_ACCURATE, cv2.RHO or cv2.LMEDS. For a simpler model, cv2.estimateAffine2D or cv2.estimateAffinePartial2D. Whichever you use, report inliers and the inlier ratio from its mask.
- **Sanity checks (recommended):** a high inlier count is not proof on its own. Consider rejecting a homography with a near-zero or negative determinant, or one whose inliers cluster in a small corner of the image.

- **Pretrained matchers (optional):** LoFTR, or ALIKED or DISK with LightGlue, are allowed if their licences permit commercial use; list each licence. SuperPoint and SuperGlue weights are licensed for non-commercial use only, so they are not allowed.

PART 2 · B

Bonus tasks

Bonus work counts only once all six requirements are done. Pick the ones you can do well rather than all of them; in a 5 to 6 hour day, one or two is plenty.

BONUS	WHAT TO SHOW
Rotation and scale	Results on the rotated and scaled pairs, and what makes your method robust (or not) to them
Partial occlusion	Results on the occluded pairs, and how much of the piece can be hidden before matching fails
SIFT vs ORB	Both on the same pairs: good matches, inliers, correct decisions and time per pair
Best match from a dataset	For each photo in `gallery_queries/`, the best match in `gallery/`, or none if nothing passes your Same threshold
Speed	Execution time and at least one optimisation, such as caching features or smaller images, with before and after times

PART 3 · SUBMIT

Commands, deliverables and how we score

PART 3 · A

Required commands

Your code must run with exactly these two commands, so we can test every submission on our hidden pairs the same way.

```
# Compare two images
python match.py --image-a data/images/img_001.jpg --image-b data/images/img_002.jpg --out output/
# prints: keypoints A and B, total matches, good matches, inliers, inlier ratio,
#         confidence (%), band (SAME / SIMILAR / DIFFERENT), time_ms
# saves:  output/img_001__img_002.jpg showing the inlier matches
```

```
# Run every pair in a pairs file
python evaluate.py --data data --pairs data/pairs.csv --out results.csv
# writes results.csv: image_a, image_b, label, pair_type, keypoints_a, keypoints_b,
#                     total_matches, good_matches, inliers, inlier_ratio,
#                     confidence, band, time_ms
# prints the results table described under Deliverables
```

HIDDEN SET

Our hidden `pairs.csv` has the same columns, but its `label` and `pair_type` values are set to `unknown`. Your `evaluate.py` must still write `results.csv` when labels are unknown; it can skip printing the metrics.

If either command fails on a fresh copy of your repository, we cannot score your hidden-set results. Test it before you submit.

PART 3 · B

Deliverables

Push everything to a Git repository and share the link, with access granted to us, before the evening deadline.

SUBMISSION CHECKLIST

- ☐ **Python source code**, including `match.py` and `evaluate.py`
- ☐ `requirements.txt`
- ☐ `README.md` explaining the approach, with setup and run steps (3 commands or fewer)
- ☐ **Sample outputs:** inlier visualisations for at least one Same, one Similar and one Different result, plus your failures
- ☐ `results.csv` from the provided pairs
- ☐ **A brief document** (`REPORT.md`, 1 to 2 pages), described below

The brief document covers:

1. **Algorithm selection:** which feature type, matcher, ratio and RANSAC settings you chose, and why.
2. **Confidence score:** your formula and how you set the three band thresholds.
3. **Accuracy observations:** a results table for the provided pairs:
   - Same pairs classified Same ÷ all same pairs
   - Different pairs classified Different or Similar ÷ all different pairs
   - Lookalike pairs wrongly classified Same (should be zero)
   - Results broken down by pair type
4. **Challenges faced:** what went wrong, with at least 2 failure examples and your explanation.
5. **Possible improvements:** what you would change or test next with more time.

PART 3 · C

Rules

Work alone, in Python, with open-source libraries only.

- **Libraries:** Python 3.10+, `opencv-python` (version 4.4 or later includes SIFT; on OpenCV 5.x install opencv-contrib-python for AKAZE and BRISK), NumPy, Matplotlib. Others are fine if listed in `requirements.txt`.
- **No model training.** This is a classical feature-matching task. A pretrained deep-learning model is optional, and only if its licence permits commercial use; list it and its licence in the brief document.
- **No cloud vision APIs or LLMs** in the matching decision. The tool must run offline.
- **No hard-coding.** Decisions must come from the images, never from file names or the labels in `pairs.csv`.
- **AI coding assistants are allowed,** but you must be able to explain and change any line of your code in the next round.
- **Individual work.** Do not share code or results with other candidates.

PART 3 · D

How we score

We score every submission out of 100 on its code, approach and results.

AREA	WHAT WE LOOK FOR
OpenCV fundamentals	All six requirements implemented correctly: right matcher and distance, ratio test, RANSAC with inlier mask
Hidden-set results	Correct decisions on pairs you have never seen
Lookalike handling	No lookalike pair classified Same, or failures clearly explained
Confidence score	A score built on inliers, with thresholds chosen from data and explained
Code quality	Runs first time from the README; clear structure, sensible functions, error handling
Brief document	Clear reasoning, correct accuracy figures, real failures and limits stated
Bonus	Quality of any bonus tasks attempted

> WHY IT MATTERS
>
> **Honest results with weaker numbers score higher than strong claims the hidden pairs do not support.**

AFTER THE DAY

What comes next

Shortlisted candidates move to the next round: a technical interview in which you walk us through your code, explain your choices and make a small change live. Come ready to explain every line.

APPENDIX

Useful References

Starting points if you need a refresher. You are free to use any other public resources.

RESOURCE	LINK
OpenCV: Introduction to SIFT	https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html
OpenCV: ORB (Oriented FAST and Rotated BRIEF)	https://docs.opencv.org/4.x/d1/d89/tutorial_py_orb.html
OpenCV: Feature Matching (BFMatcher, FLANN, ratio test)	https://docs.opencv.org/4.x/dc/dc3/tutorial_py_matcher.html
OpenCV: Feature Matching + Homography to find Objects	https://docs.opencv.org/4.x/d1/de0/tutorial_py_feature_homography.html
Lowe (2004): Distinctive Image Features from Scale-Invariant Keypoints, the SIFT paper	https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf